

Universidad Nacional Autónoma de México

Facultad de Ciencias



Reporte de Servicio Social

Aplicación de dos técnicas de Machine Learning
-árboles de decisión y bosques aleatorios- para la
clasificación de exoplanetas usando datos de la
misión Kepler de la NASA



Facultad de Ciencias, Universidad Nacional
Autónoma de México, Ciudad Universitaria,
Circuito Exterior SN, C.P. 04510

17 de octubre de 2025

Jesús Alberto Cerón Hernández

Resumen

En este trabajo apliqué y comparé dos técnicas de **aprendizaje automático supervisado** —*árboles de decisión* y *bosques aleatorios*— para la **clasificación de candidatos a exoplanetas** utilizando los datos del catálogo *Kepler Object of Interest (KOI)* de la misión **Kepler** de la NASA. El conjunto de datos que empleé contiene más de 9,000 registros y 49 características. Para su análisis, diseñé un *pipeline* que incluyó limpieza, imputación de valores faltantes, selección de variables con **SelectKBest**, ponderación de clases y validación cruzada, asegurando un flujo de trabajo reproducible y sin sesgos.

Desarrollé dos enfoques de clasificación: uno **binario**, que distingue entre planetas confirmados y falsos positivos, y otro **multiclase**, que también incluye a los candidatos planetarios. En el modelo binario, el *Bosque Aleatorio Regularizado* alcanzó una *accuracy* del **93.0 %** y un *F1-score* del **92.9 %**. En el modelo multiclase, obtuvo una *accuracy* del **75.1 %** y un *F1-score* del **73.5 %**, superando en ambos casos al árbol de decisión y a las redes neuronales, en cuanto a valores de las métricas y con un mejor valor para el gap de overfitting. A partir del análisis de importancia de características, observé que las variables más relevantes fueron el **radio planetario** (*koi_prad*), la **temperatura efectiva estelar**, la **duración del tránsito** y la **profundidad de la señal**.

Con lo cual llegué a que los **métodos de ensemble**, especialmente el *Bosque Aleatorio*, ofrecen una combinación óptima de precisión, estabilidad y capacidad de generalización, lo que los convierte en una herramienta poderosa para la **clasificación automática de exoplanetas**. Este trabajo demuestra el potencial del **Machine Learning** para apoyar la investigación astronómica moderna y la creciente disciplina de la **astroinformática**.

Palabras clave: Kepler, exoplanetas, Machine Learning, árboles de decisión, bosques aleatorios, clasificación multiclase, astroinformática.

Código Fuente: El código completo del proyecto está disponible en el repositorio de GitHub:
<https://github.com/ceronbuu/Clasificacion-de-Exoplanetas-con-Machine-Learning-y-Datos-Kepler>.



Índice

1	Introducción	2
1.0.1	El método del tránsito	3
1.0.2	El Satélite, su operación y el pipeline de Datos	4
1.0.3	Resultados científicos y legado	6
1.1	Métodos utilizados de Machine Learning en la clasificación astronómica	6
1.1.1	Árboles de decisión	7
1.1.2	Bosques aleatorios	10
1.2	Métricas de evaluación para clasificación	11
1.2.1	La Matriz de Confusión	12
1.2.2	Métricas derivadas de la matriz de confusión	13
1.3	El Problema del desbalance de clases	14
1.3.1	Estrategias de mitigación	14
1.4	Validación de modelos y prevención del sobreajuste	15
1.4.1	Train-Test split	15
1.4.2	La Robustez de la validación cruzada (Cross-validation)	15
2	Proyecto y planificación	16
2.1	Objetivos	16
2.2	Metodología de trabajo	17
2.2.1	Identificación de actividades	17
3	Materiales y métodos	18
3.1	Conjunto de datos	18
3.2	Framework y Librerías de Software	18
3.2.1	Pandas	18
3.2.2	Scikit-learn	18
3.2.3	Matplotlib & Seaborn	19
3.3	Metodología	19
3.3.1	Pipeline en aprendizaje automático	19
3.3.2	Ejemplo de transformaciones comunes	20
3.3.3	Tratamiento de los datos	21
3.4	Resultados y discusión	22
3.4.1	Forma binaria.	23
3.4.2	Forma multiclase.	25
3.4.3	Resultados finales	27
4	Conclusión	28
	Bibliografía	29

1 Introducción

Desde el desplazamiento de la Tierra del centro del universo por la revolución copernicana, hemos aprendido que nuestra existencia se enmarca en un contexto vasto y, en gran medida, inexplorado. El estudio de los exoplanetas (planetas que orbitan estrellas distintas a nuestro Sol) representa la frontera contemporánea de esta búsqueda. Mi proyecto de servicio social se sumerge en esta disciplina, abordando un desafío que no es de naturaleza observacional, sino computacional, con el desafío de la interpretación de datos a una escala masiva.

La misión kepler de la NASA ha sido una de las misiones más importantes para conocer nuestro universo. Antes de su lanzamiento, el descubrimiento de exoplanetas era un proceso artesanal; hoy, es una industria de datos. kepler nos dio un archivo público con las curvas de luz de cientos de miles de estrellas. Por cada candidato planetario prometedor, el archivo contiene una multitud de **falsos positivos**: fenómenos astrofísicos y artefactos instrumentales que actúan como impostores (Mullally et al., 2015).

La validación manual de estos candidatos, aunque rigurosa, se ha vuelto insostenible. La hipótesis fundamental es que la clasificación de candidatos a exoplanetas puede ser tratada como un problema de reconocimiento de patrones. El objetivo de mi trabajo fue, por tanto, diseñar, implementar y evaluar un pipeline de clasificación basado en dos algoritmos más importantes de la ciencia de datos: los **árboles de decisión**, por su interpretabilidad, y los **bosques aleatorios**, por su robustez y precisión superior.

2.1 La misión Kepler.

El contexto pre-Kepler y el dominio de la velocidad radial

Antes de que mi trabajo con los datos de Kepler fuera siquiera posible, el campo de los exoplanetas estaba apenas comenzando, dominado por una técnica ingeniosa pero inherentemente sesgada: el método de la velocidad radial. Para entender la revolución que supuso Kepler, primero tuve que comprender a fondo los logros y las limitaciones del método que lo precedió.

El principio de la velocidad radial se basa en el movimiento entre una estrella y sus planetas. No es del todo correcto decir que un planeta orbita a su estrella; en realidad, ambos cuerpos orbitan su centro de masa común o baricentro. Como la estrella es inmensamente más masiva, el baricentro se encuentra muy cerca del centro de la estrella, a menudo dentro de la propia estrella. Sin embargo, esto provoca que la estrella ejecute un pequeño “bamboleo” orbital. Desde nuestra perspectiva en la Tierra, este bamboleo se manifiesta como un movimiento periódico de la estrella, acercándose y alejándose de nosotros.

Este movimiento es detectable gracias al efecto Doppler. Cuando la estrella se mueve hacia nosotros, la longitud de onda de su luz se comprime ligeramente, desplazándose hacia el azul (blueshift). Cuando se aleja, la longitud de onda se estira, desplazándose hacia el rojo (redshift). Al medir con espectrógrafos de alta precisión estas minúsculas y periódicas variaciones en el espectro de la estrella, podemos inferir la presencia de un compañero invisible y deducir su período orbital y una estimación de su masa.

Sin embargo, este método, que nos dio los primeros descubrimientos de exoplanetas, sufre de desventajas críticas que moldearon nuestra visión inicial del universo y que hicieron necesaria una misión como Kepler:

- **Ambigüedad de la masa:** La técnica solo mide la velocidad a lo largo de nuestra línea de visión. Esto significa que no podemos determinar la masa real del planeta (M_p), sino solo

su masa mínima, un valor conocido como $M_p \sin(i)$, donde i es la inclinación de la órbita del planeta con respecto a nosotros. Si la órbita es perfectamente de canto ($i = 90^\circ$), el valor medido es la masa real. Pero si la órbita es vista de frente ($i = 0^\circ$), no hay movimiento radial y el planeta es completamente indetectable.

- **Sesgo de selección fundamental:** El movimiento estelar es mucho más pronunciado (y por lo tanto, más fácil de detectar) para planetas muy masivos y para aquellos en órbitas muy cercanas a su estrella. Esto no es una limitación menor; es un sesgo abrumador. Como resultado, los primeros catálogos de exoplanetas se llenaron de "Júpiteres calientes", mundos gigantes gaseosos en órbitas increíblemente cortas. Esto pintó una imagen extraña y posiblemente engañosa de los sistemas planetarios y dejó sin respuesta la pregunta más profunda: ¿son los planetas pequeños y rocosos como la Tierra una rareza o una norma en la galaxia?
- **No proporciona el radio:** El método de velocidad radial no nos dice nada sobre el tamaño físico del planeta. Sin el radio, no podemos calcular su densidad, un parámetro fundamental para distinguir entre un mundo rocoso, uno gaseoso o uno helado.

Fue evidente para la comunidad científica que, para responder a la pregunta sobre la frecuencia de planetas como la Tierra, necesitábamos un enfoque diferente. No bastaba con una búsqueda dirigida a las señales más fuertes, sino que se requería un censo estadístico imparcial, capaz de detectar planetas más pequeños y de determinar sus radios. Esta necesidad fue la semilla de la misión Kepler y la razón por la que se adoptó un método completamente distinto, el cual se explicaría a continuación.

1.0.1 El método del tránsito

Fundamentos físicos y geometría del tránsito. Se basa en un principio simple: la ocultación. Si la órbita de un planeta está alineada con mi línea de visión (una configuración *edge-on*), pasará por delante del disco de su estrella, bloqueando una pequeña fracción de su luz. Esta atenuación, aunque minúscula, es medible. La profundidad relativa del tránsito, $\Delta F/F$, está directamente relacionada con la razón de las áreas: $\Delta F/F \approx (R_p/R_*)^2$. Para que un planeta del tamaño de la Tierra transite una estrella similar al Sol, la caída de brillo es de apenas 84 partes por millón (ppm), lo que es un desafío fotométrico extremo.

La duración del tránsito, T_{dur} , no solo depende del período orbital y el radio estelar, sino también del **parámetro de impacto** (b), que define la latitud del tránsito en el disco estelar ($b = 0$ para un tránsito ecuatorial, $b = 1$ para un tránsito rasante por el polo). Un tránsito con un alto parámetro de impacto es más corto que uno ecuatorial. La forma detallada del ingreso y egreso (los "hombros" de la curva de luz) está modulada por el **oscurecimiento del limbo**, un efecto óptico donde el borde del disco estelar aparece más oscuro que el centro. El modelado preciso de esta forma es una herramienta poderosa, ya que las estrellas binarias eclipsantes a menudo producen curvas de luz con forma de "V", mientras que los planetas, al ser cuerpos opacos, tienden a producir tránsitos con un fondo plano en forma de U (Mandel & Agol, 2002).

El Desafío crítico de los falsos positivos. La principal dificultad del método de tránsito, es la abrumadora tasa de falsos positivos. Un algoritmo de búsqueda simple puede identificar miles de señales periódicas, pero la gran mayoría no son planetas. La validación requiere un análisis minucioso para descartar múltiples escenarios astrofísicos e instrumentales:

- **Sistemas de binarias eclipsantes (EBs):** Son la principal fuente de contaminación. Este escenario ocurre cuando dos estrellas se orbitan y eclipsan mutuamente. Si el eclipse

es rasante (parcial), o si una de las estrellas es significativamente más pequeña o más fría que la principal (por ejemplo, una enana M orbitando una estrella tipo Sol), la caída de brillo puede ser lo suficientemente pequeña como para imitar la de un planeta gigante gaseoso. La detección de un eclipse secundario, por pequeño que sea, es una prueba casi irrefutable de que el compañero es auto-luminoso (una estrella) y no un planeta.

- **Binarias eclipsantes de fondo (BEBs):** Este es el tipo de falso positivo más insidioso y difícil de identificar. Ocurre cuando una estrella binaria eclipsante, que no está físicamente asociada a la estrella objetivo, se encuentra casualmente dentro de la misma apertura fotométrica del detector. La luz de la estrella objetivo, que es mucho más brillante y constante, se mezcla con la luz de la BEB. Esto tiene el efecto de **diluir** la señal de eclipse de la BEB. Por ejemplo, un profundo eclipse del 50 % de una BEB muy débil puede ser diluido por la luz de la estrella objetivo hasta parecer un tránsito planetario del 0.1 %. La herramienta más poderosa para desenmascarar a las BEBs es el **análisis de centroide**: durante un evento causado por una fuente de fondo, el centro de luz combinado de la apertura se desplazará sutilmente en la dirección de esa fuente.
- **Variabilidad estelar intrínseca:** La rotación de grandes grupos de manchas estelares (regiones más frías y oscuras en la superficie, análogas a las manchas solares) puede causar modulaciones periódicas en la curva de luz. Aunque estas variaciones suelen tener una forma más sinusoidal y menos abrupta que un tránsito, pueden ser lo suficientemente complejas como para engañar a los algoritmos de búsqueda. Otras formas de variabilidad, como las pulsaciones estelares (oscilaciones en el tamaño y temperatura de la estrella), también pueden introducir ruido periódico.

1.0.2 El Satélite, su operación y el pipeline de Datos

Diseño Instrumental y Estrategia Observacional. El satélite constituye un instrumento astronómico de vanguardia, concebido específicamente para la detección de exoplanetas mediante fotometría de alta precisión. Su diseño incorpora un telescopio Schmidt con una apertura de 0.95 metros y una cámara CCD de 95 megapíxeles, capaz de cubrir un amplio campo de visión de aproximadamente 115 grados cuadrados centrado en la constelación de Cygnus. Este enfoque permitió observar simultáneamente decenas de miles de estrellas, aumentando la probabilidad de detectar tránsitos planetarios incluso en estrellas de brillo relativamente débil.

La estrategia de observación adoptada, conocida como “mirada fija” (*staring mode*), consistió en mantener un campo estelar constante durante períodos prolongados. Esta técnica es fundamental para establecer una línea de base temporal extensa, necesaria para detectar múltiples tránsitos y confirmar la periodicidad orbital de los planetas, incluyendo aquellos con períodos largos que requieren observaciones continuas durante meses. La estabilidad de la orientación del satélite, garantizada por un sistema de giroscopios y ruedas de reacción de alta precisión, permitió mantener la misma región del cielo dentro del campo de visión con desviaciones mínimas.

Además, se implementaron estrategias de mitigación de ruido instrumental y ambiental. La calibración periódica de los detectores, combinada con la corrección de variaciones térmicas y radiación de fondo, permitió mantener la precisión fotométrica necesaria para detectar cambios de brillo estelares del orden de unas pocas partes por millón. Esto es esencial para la detección de planetas tipo terrestre, cuyo tránsito genera atenuaciones extremadamente pequeñas en la luz de la estrella anfitriona.

Operación del Satélite. El satélite operó en órbita terrestre alta, minimizando la interferencia de la atmósfera y la luz zodiacal. La telemetría constante permitió monitorear la salud de los sistemas a bordo y ajustar parámetros operativos en tiempo real. El control de actitud y

la compensación de microvibraciones fueron críticos para garantizar que las imágenes fueran nítidas y estables, evitando que el desenfoque o el movimiento comprometeran la calidad de los datos.

La planificación de la misión incluyó campañas de observación secuenciales, en las que el satélite podía mantener la misma región del cielo durante varios meses consecutivos, optimizando la acumulación de datos y asegurando que los tránsitos de períodos largos fueran capturados de manera consistente. Los datos fueron almacenados a bordo en sistemas de memoria de alta capacidad antes de ser transmitidos a estaciones terrestres, asegurando que se minimizaran pérdidas por interrupciones en la comunicación.



Figura 1: Telescopio kepler

El pipeline de procesamiento científico. Los datos crudos de son inutilizables sin un procesamiento intensivo. El procesamiento que hicieron en el Centro de investigación Ames de la Nasa fue el siguiente:

- **CAL (Calibración):** Corrige los efectos sistemáticos de bajo nivel en los detectores CCD, como el bias, la corriente oscura y las variaciones de sensibilidad de los píxeles (flat-fielding).
- **PA (Análisis Fotométrico):** Utiliza fotometría de apertura para extraer el flujo de cada estrella, generando una curva de luz cruda.
- **PDC (Acondicionamiento de Datos Pre-búsqueda):** El algoritmo PDC-MAP (Maximum A Posteriori) identifica y elimina el ruido instrumental sistemático (causado por

variaciones térmicas, vibraciones, etc.) que es común a muchas estrellas. Lo hace construyendo una base de "vectores de co-tendencia.^a partir de un conjunto de estrellas silenciosas y ajustándolos a la curva de luz de cada estrella para eliminar el ruido sin distorsionar las señales astrofísicas reales.

- **TPS (Búsqueda de Planetas en Tránsito):** Este módulo utiliza un algoritmo de "filtro adaptado" para buscar señales periódicas con forma de caja en cada curva de luz. Examina miles de millones de combinaciones de período, fase y duración para cada estrella. Cuando una señal supera un umbral estadístico de significancia (típicamente 7.1σ), se etiqueta como un Evento de Umbral de Tránsito (TCE).
- **DV (Validación de Datos):** Cada TCE es sometido a un análisis exhaustivo. El DV ajusta un modelo de tránsito completo a la curva de luz y realiza una batería de pruebas diagnósticas, generando docenas de métricas. Estas métricas incluyen la significancia del desplazamiento del centroide, la diferencia de profundidad entre tránsitos pares e impares (para buscar eclipses secundarios), y la forma de la señal. El informe de DV es el producto final que sirve como el **vector de características** para mis modelos de Machine Learning.

1.0.3 Resultados científicos y legado

Demografía Planetaria y la Brecha de Fulton. kepler transformó la exoplanetología de una ciencia de descubrimientos individuales a una de estadística de poblaciones. Su resultado más profundo fue la confirmación de que los planetas pequeños son extraordinariamente comunes. El descubrimiento más sorprendente fue la "brecha de radio" o "Fulton Gap", una marcada escasez de planetas con radios entre 1.5 y 2.0 veces el de la Tierra (Fulton et al., 2017). La explicación más aceptada para esta brecha es la **fototevaporación**: la intensa radiación de alta energía de una estrella joven puede despojar las atmósferas primordiales de hidrógeno y helio de los planetas de tamaño intermedio. Los planetas que comienzan justo por debajo de un umbral de masa crítico pierden toda su atmósfera y se convierten en "Super-Tierras rocosas", mientras que aquellos justo por encima del umbral logran retener sus atmósferas y se convierten en "Mini-Neptunos" gaseosos. La brecha representa la zona de transición inestable entre estos dos regímenes evolutivos.

La Misión Extendida K2. En 2013, un segundo de los cuatro volantes de reacción del satélite falló. Esto impidió el apuntado de alta precisión necesario para la misión principal. Sin embargo, gracias al ingenio de los ingenieros, se desarrolló un nuevo modo de operación que utilizaba la presión de la radiación solar como un "tercer volante de reacción virtual" para estabilizar el telescopio. Esto dio lugar a la misión extendida K2. En esta misión, observó diferentes campos a lo largo de la eclíptica durante campañas de aproximadamente 80 días cada una. Aunque la precisión fotométrica era menor, K2 exploró una mayor diversidad de entornos galácticos, descubriendo planetas alrededor de estrellas en cúmulos estelares jóvenes y de diferentes tipos espectrales.

1.1 Métodos utilizados de Machine Learning en la clasificación astronómica

El éxito de cualquier modelo de aprendizaje supervisado depende críticamente de la calidad de sus características de entrada. El informe de DV del pipeline de es una mina de oro de características diseñadas por expertos, que yo utilizo, como por ejemplo:

- `'koi__period'`, `'koi__depth'`, `'koi__duration'`: Las propiedades fundamentales del tránsito.
- `'koi__model_nr'`: La relación señal-ruido del modelo de tránsito ajustado. Es una medida de la robustez de la detección.
- `'koi__fpflag_ss'` (*StellarEclipseFlag*): Una bandera que se activa si la forma de la señal es más parecida a una “V” que a una “U”, o si se detecta un eclipse secundario significativo.
- `'koi__fpflag_co'` (*CentroidOffsetFlag*): Una bandera que se activa si el análisis del centroide indica un desplazamiento estadísticamente significativo durante el tránsito, un fuerte indicador de una BEB.
- `'koi__prad'` (*PlanetRadius*): El radio planetario estimado, una de las características más predictivas.
- `'koi__steff'`, `'koi__srad'`: La temperatura y el radio de la estrella anfitriona.

1.1.1 Árboles de decisión

Analogía conceptual y definición formal. Un árbol de decisión puede ser imaginado como una versión automatizada y optimizada del juego de 20 "Preguntas". El algoritmo aprende por sí mismo cuál es la secuencia de preguntas más eficiente para clasificar un objeto desconocido. Para mi problema, el objeto es un candidato a exoplaneta, y las preguntas son sobre sus características medibles (ej. 'Es la profundidad del tránsito mayor a 500 ppm?', 'Tiene la curva de luz una forma en V?'). Formalmente, un árbol de decisión es un modelo de aprendizaje supervisado, no paramétrico, que aprende una función que mapea las características de una observación a una etiqueta de clase de salida mediante una serie de reglas de decisión jerárquicas. Su principal atractivo en un contexto científico es su naturaleza de "caja blanca" el modelo final es completamente transparente e interpretable.

La Anatomía de un Árbol. Para comprender su funcionamiento, me parece esencial definir su estructura:

- **Nodo raíz (Root node):** Es el punto de partida del árbol, el nodo superior que contiene el conjunto de datos de entrenamiento completo.
- **Nodos internos (Internal nodes):** Cada nodo interno representa una “pregunta” o una prueba sobre una única característica. Esta prueba divide los datos que llegan al nodo en dos o más subconjuntos.
- **Ramas (Branches):** Son las conexiones entre nodos, representando el resultado de la prueba. En un árbol de decisión binario (el tipo más común), cada nodo interno tiene dos ramas, típicamente etiquetadas como “Verdadero” y “Falso”.
- **Nodos Hoja (Leaf nodes o terminal nodes):** Son los nodos finales del árbol que no se dividen más. Cada hoja representa una predicción de clase. Una vez que una instancia de datos llega a una hoja, se le asigna la clase mayoritaria de las instancias de entrenamiento que terminaron en esa misma hoja.

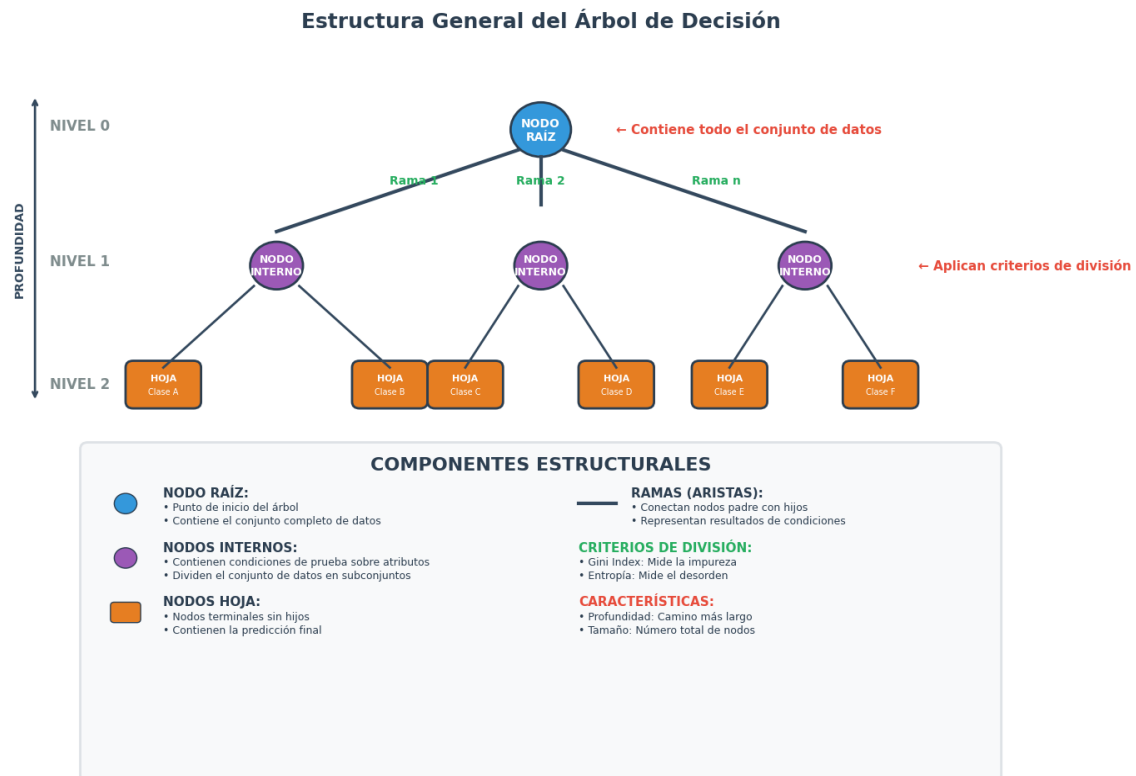


Figura 2: Arbol de decisión.

El algoritmo de construcción: particionamiento recursivo. Los árboles se construyen mediante un proceso conocido como particionamiento recursivo, un algoritmo de tipo "divide y vencerás" (como los famosos algoritmos ID3, C4.5 y CART). El proceso se puede resumir en los siguientes pasos:

1. **Inicio:** El algoritmo comienza en el nodo raíz con todo el conjunto de datos de entrenamiento.
2. **Búsqueda de la mejor división:** El algoritmo itera a través de todas las características disponibles. Para cada característica, evalúa todos los posibles puntos de corte (umbrales) para dividir los datos.
3. **Evaluación de divisiones:** Para cada posible división, el algoritmo calcula una métrica de "pureza" o "impureza" (como Gini o Entropía) en los subconjuntos resultantes (los nodos hijos). El objetivo es encontrar la división (la combinación de característica y umbral) que produzca la mayor "ganancia" o, de forma equivalente, la menor impureza ponderada en los nodos hijos.
4. **Creación de la división:** Una vez identificada la mejor división, el algoritmo la implementa, creando un nodo interno con esa regla de decisión y dos nuevos nodos hijos. Los datos del nodo padre se reparten entre los hijos según si cumplen o no la regla.
5. **Recursividad:** El algoritmo repite los pasos 2, 3 y 4 para cada uno de los nuevos nodos hijos de forma independiente.

6. **Criterios de parada:** La recursión se detiene para una rama cuando se cumple una de varias condiciones predefinidas (hiperparámetros): el nodo resultante es perfectamente puro (todas las instancias pertenecen a la misma clase); se alcanza la profundidad máxima permitida para el árbol; el número de instancias en un nodo es inferior a un umbral mínimo para ser dividido; o no se puede encontrar una división que mejore la pureza de manera significativa.

Criterios de división. La selección de la mejor división en cada nodo es el núcleo del algoritmo. Los dos criterios más utilizados son:

- **Impureza de gini:** Esta métrica mide la probabilidad de clasificar incorrectamente una instancia si fuera etiquetada aleatoriamente según la distribución de clases en el nodo. La fórmula para un conjunto S es:

$$G(S) = 1 - \sum_{i=1}^c p_i^2 \quad (1)$$

donde p_i es la proporción de instancias de la clase i en el nodo. Un valor de Gini de 0 representa una pureza perfecta (todas las instancias son de una sola clase), mientras que un valor de 0.5 (para un problema de dos clases) representa la máxima impureza. El algoritmo busca la división que minimice la impureza Gini ponderada de los nodos hijos.

- **Ganancia de información (Information gain):** Este criterio, basado en la teoría de la información, utiliza la entropía para medir el desorden o la incertidumbre en un nodo. La entropía se calcula como:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (2)$$

La ganancia de información es la reducción en la entropía que se logra al dividir los datos. El algoritmo elige la división que maximiza esta ganancia, es decir, la que reduce más la incertidumbre sobre la clase de las instancias.

El Dilema del sobreajuste (Overfitting) y la poda (Pruning). La mayor debilidad de un árbol de decisión individual es su tendencia al **sobreajuste**. Si se le permite crecer sin restricciones, el árbol puede volverse excesivamente complejo, creando una regla para cada instancia individual en los datos de entrenamiento. Este modelo se “aprende de memoria” el ruido y las peculiaridades del conjunto de entrenamiento, pero pierde su capacidad de generalizar a nuevos datos no vistos. Para combatir esto, yo empleo técnicas de **poda (pruning)**:

- **Pre-poda (Early Stopping):** Consiste en detener el crecimiento del árbol antes de que se vuelva demasiado complejo. Esto se logra estableciendo hiperparámetros como ‘*max_depth*’ (la profundidad máxima del árbol), ‘*min_samples_split*’ (el número mínimo de instancias que debe tener un nodo para poder ser dividido) o ‘*min_samples_leaf*’ (el número mínimo de instancias que debe tener un nodo hoja).
- **Post-poda (Pruning):** Consiste en dejar que el árbol crezca completamente y luego “recortar” las ramas que aportan poco poder predictivo. Un método común es la Poda por Complejidad de Costo, que busca un equilibrio óptimo entre la complejidad del árbol (el número de hojas) y su error de clasificación.

1.1.2 Bosques aleatorios

Teoría de ensambles y el trade-off de Sesgo-Varianza. Los Bosques Aleatorios como uno de los ejemplos más exitosos de **aprendizaje por conjuntos (ensemble learning)**, una meta-estrategia en machine learning que se basa en elegir bien entre tantas opciones. El error de un modelo predictivo puede descomponerse en tres componentes: sesgo, varianza y ruido irreducible. El **sesgo** es el error debido a suposiciones simplificadoras del modelo (subajuste). La **varianza** es el error debido a la sensibilidad del modelo a pequeñas fluctuaciones en los datos de entrenamiento (sobreajuste). Los árboles de decisión individuales, si se dejan crecer profundos, son modelos de bajo sesgo (pueden capturar estructuras complejas) pero de muy alta varianza. La idea fundamental de los bosques aleatorios es construir cientos de estos modelos de alta varianza de una manera específica y promediar sus predicciones. Matemáticamente, el promediado de estimadores no correlacionados reduce drásticamente la varianza general del modelo final, lo que resulta en un clasificador que mantiene el bajo sesgo de los árboles individuales pero con una varianza mucho menor, lo que lo hace extremadamente robusto y preciso.

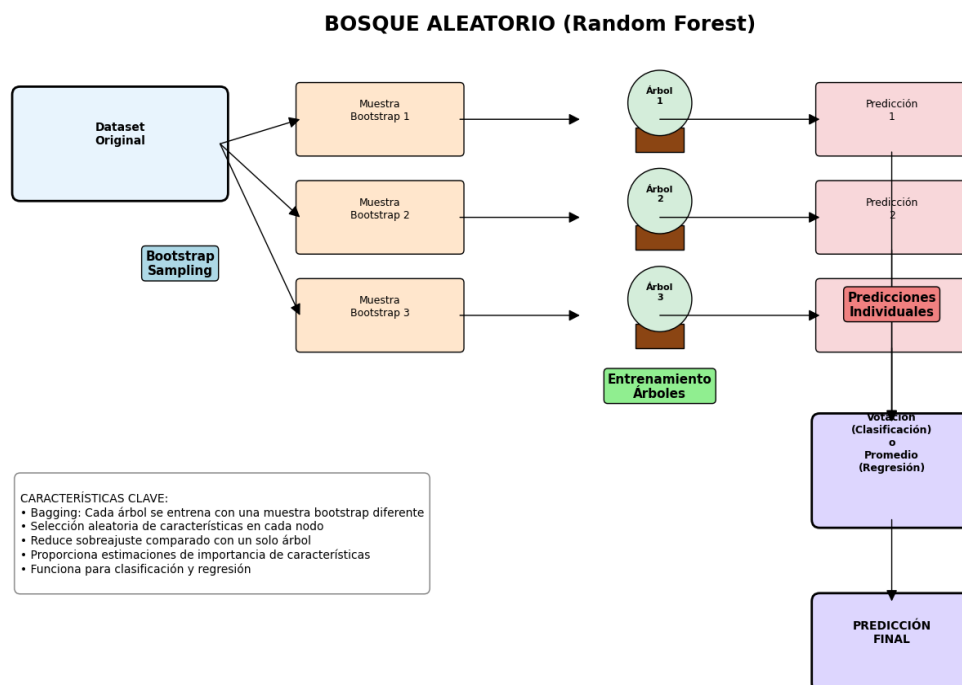


Figura 3: Bosques Aleatorios.

Partes importantes de la aleatorización. La clave para que un bosque aleatorio funcione es que los árboles que lo componen sean lo más diversos y no correlacionados posible. El algoritmo logra esto mediante dos poderosos mecanismos de aleatorización (Breiman, 2001):

- **Bagging (Bootstrap aggregating):** Bagging es el mecanismo principal para generar diversidad en los datos de entrenamiento de cada árbol. En lugar de entrenar cada árbol con el mismo conjunto de datos, el algoritmo crea una "muestra bootstrap" para cada uno. Una muestra bootstrap es un nuevo conjunto de datos del mismo tamaño que el original, pero

creado mediante muestreo aleatorio *con reemplazo*. Esto significa que algunas instancias originales pueden aparecer varias veces en la muestra de un árbol, mientras que otras (en promedio, alrededor del 36.8%) no aparecerán en absoluto. Al entrenar cada árbol en una versión ligeramente diferente de los datos, cada uno aprende patrones ligeramente distintos, lo que contribuye a su diversidad.

- **Aleatoriedad de características (Feature subsampling):** Este es el ingrediente secreto que distingue a los bosques aleatorios de otros métodos de bagging. Si hubiera una característica muy predictiva en el conjunto de datos, la mayoría de los árboles de un ensamble de bagging la elegirían como la primera división en su nodo raíz, lo que haría que todos los árboles fueran muy similares (altamente correlacionados), anulando los beneficios del promediado. Para evitar esto, al construir cada nodo de cada árbol, el algoritmo no considera todas las características disponibles para encontrar la mejor división. En su lugar, selecciona un **subconjunto aleatorio** de características (un hiperparámetro común es $\sqrt{N_{\text{features}}}$). Esto fuerza a los árboles a explorar una variedad mucho más amplia de características predictivas, incluso las más débiles, lo que resulta en árboles mucho más diversos y decorrelacionados.

Mecanismos internos: votación, error OOB y feature importance. El funcionamiento final del bosque se basa en tres conceptos:

- **Agregación por votación:** Una vez que los cientos de árboles han sido entrenados, la predicción para una nueva instancia se realiza pasándola a través de todos los árboles del bosque. Cada árbol emite una predicción (un "voto"). Para problemas de clasificación, la predicción final del bosque es la clase que recibe la mayoría de los votos (lo que se conoce como "hard voting").
- **Error out-of-bag (OOB):** El proceso de bagging ofrece una forma elegante y computacionalmente eficiente de estimar el error de generalización del modelo sin necesidad de un conjunto de prueba separado. Para cada instancia en el conjunto de datos original, puedo identificar todos los árboles que *no* la utilizaron en su muestra bootstrap (su conjunto "Out-of-Bag"). Puedo usar estos árboles para hacer una predicción sobre esa instancia. Al hacer esto para todas las instancias, obtengo una predicción OOB para todo el conjunto de datos, y el error de estas predicciones (el "error OOB") sirve como una estimación no sesgada y robusta del rendimiento del modelo en datos nuevos.
- **Importancia de características (Feature importance):** Los bosques aleatorios proporcionan una métrica muy útil para clasificar la importancia de las características. El método más común es la "importancia Gini" o "disminución media de la impureza". Para cada característica, el algoritmo calcula cuánto reduce esa característica la impureza de Gini en promedio, cada vez que se utiliza para una división en todos los árboles del bosque. Las características que consistentemente producen las mayores reducciones de impureza son consideradas las más importantes. Esto me permite abrir la "caja negra" del modelo y entender qué parámetros astrofísicos son los más decisivos para la clasificación.

1.2 Métricas de evaluación para clasificación

Introducción a la Evaluación de Modelos. La construcción de un modelo de clasificación no termina con el proceso de entrenamiento. De hecho, una de las etapas más críticas en el ciclo de vida del desarrollo de un modelo de *Machine Learning* es su evaluación cuantitativa. No es

suficiente con que un modelo haga predicciones; necesito saber con precisión qué tan buenas son esas predicciones y, lo que es más importante, entender la naturaleza de sus errores. Seleccionar la métrica de evaluación correcta es tan crucial como seleccionar el algoritmo correcto, especialmente en problemas complejos como la clasificación de exoplanetas, donde los costos de los diferentes tipos de errores no son simétricos. Una métrica inadecuada puede llevar a conclusiones engañosas sobre el rendimiento de un modelo. Por ejemplo, un modelo que parece muy preciso a primera vista podría ser completamente inútil en la práctica si no es evaluado a través del lente correcto.

1.2.1 La Matriz de Confusión

Definición y Estructura. La base para comprender el rendimiento de un clasificador es la **matriz de confusión**. No es una métrica en sí misma, sino una tabla que desglosa y resume el resultado de las predicciones del modelo en comparación con las etiquetas reales (la "verdad terreno"). Para entenderlos mejor, por ejemplo, para mi problema, que es de clasificación binaria entre "Planeta Genuino" (la clase positiva) y "Falso Positivo" (la clase negativa), la matriz tiene cuatro cuadrantes:

- **Verdaderos positivos (TP - True positives):** Instancias que eran realmente planetas y mi modelo predijo correctamente como planetas. **Este es el objetivo principal de la ciencia:** un descubrimiento exitoso.
- **Verdaderos negativos (TN - True negatives):** Instancias que eran realmente falsos positivos y mi modelo predijo correctamente como falsos positivos. **Este es un resultado deseable:** mi modelo ha filtrado con éxito el ruido.
- **Falsos positivos (FP - False positives) - Error de Tipo I:** Instancias que eran falsos positivos, pero mi modelo predijo erróneamente como planetas. **Este es un error costoso:** cada falso positivo consume tiempo valioso de seguimiento con otros telescopios y recursos computacionales, persiguiendo una señal que no es real.
- **Falsos negativos (FN - False negatives) - Error de Tipo II:** Instancias que eran planetas reales, pero mi modelo predijo erróneamente como falsos positivos. **Este es el peor error desde la perspectiva del descubrimiento:** un planeta potencial es descartado y se pierde la oportunidad de estudiarlo.

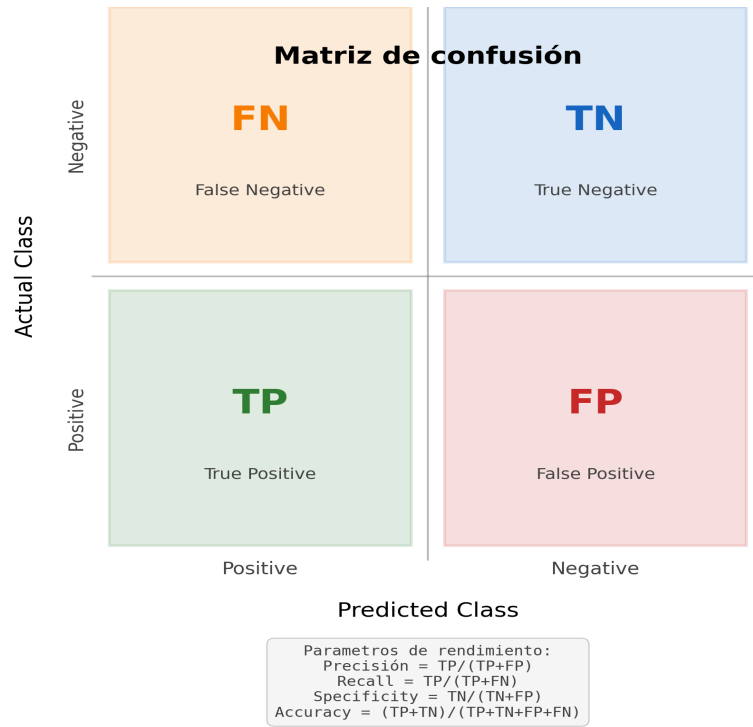


Figura 4: Matriz de confusión.

1.2.2 Métricas derivadas de la matriz de confusión

Exactitud (Accuracy). Es la métrica más intuitiva. Mide la proporción de predicciones correctas sobre el total de predicciones.

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

Sin embargo, la exactitud es notoriamente engañosa en conjuntos de datos con **clases desbalanceadas**, como el de . Imagina un conjunto de datos donde el 95 % de los candidatos son Falsos Positivos y solo el 5 % son Planetas Genuinos. Un modelo trivial que simplemente prediga "Falso Positivo" para cada instancia lograría una exactitud del 95 %, pero sería completamente inútil porque nunca descubriría un solo planeta. Esto se conoce como la **paradoja de la exactitud**.

Precisión (Precision). La precisión responde a la pregunta: "De todas las instancias que etiqueté como planetas, ¿cuántas eran realmente planetas?". Es la métrica de la **confiabilidad**. Un modelo con alta precisión genera pocas alarmas falsas. En mi contexto, una alta precisión es vital para asegurar que los recursos de seguimiento no se desperdicien en falsos positivos.

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (4)$$

Sensibilidad (Recall o exhaustividad). La sensibilidad (o *recall*) responde a la pregunta: "De todos los planetas reales que había en el conjunto de datos, ¿cuántos fui capaz de encontrar?". La sensibilidad es la métrica del **descubrimiento**. Un modelo con alta sensibilidad es bueno

para no dejar pasar ningún planeta real, ya que minimiza los falsos negativos.

$$\text{Sensibilidad} = \frac{TP}{TP + FN} \quad (5)$$

Puntuación F1 (F1-Score). En la mayoría de los problemas del mundo real, existe un compromiso (un *trade-off*) entre la precisión y la sensibilidad. Un modelo muy “cauteloso” puede tener una alta precisión pero una baja sensibilidad (solo clasifica como planeta los casos más obvios), mientras que un modelo muy “agresivo” puede tener alta sensibilidad pero baja precisión (encuentra casi todos los planetas pero también clasifica muchos falsos positivos como planetas). La puntuación F1 es la **media armónica** de ambas métricas:

$$\text{Puntuación F1} = 2 \times \frac{\text{Precisión} \times \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}} \quad (6)$$

La media armónica penaliza fuertemente los valores extremos. Por lo tanto, una puntuación F1 alta sólo se logra cuando tanto la precisión como la sensibilidad son altas, lo que la convierte en una métrica excelente para evaluar el rendimiento general de un clasificador en problemas con clases desbalanceadas.

1.3 El Problema del desbalance de clases

Definición e impacto en el entrenamiento. Un conjunto de datos está desbalanceado cuando el número de instancias de una clase es significativamente mayor que el de las otras. Este es el estado natural de muchos problemas de detección en ciencia (detección de fraudes, diagnóstico médico, y por supuesto, búsqueda de exoplanetas), donde el evento de interés es, por definición, raro. El conjunto de datos de es un ejemplo canónico, donde la clase ‘FALSE POSITIVE’ es abrumadoramente mayoritaria.

El principal problema es que la mayoría de los algoritmos de Machine Learning están diseñados para optimizar métricas como la exactitud. En un escenario desbalanceado, el modelo aprende rápidamente que puede lograr una alta exactitud simplemente prediciendo siempre la clase mayoritaria. Esto resulta en un modelo con un **sesgo** intrínseco, que efectivamente ignora a la clase minoritaria y, por lo tanto, falla en su propósito fundamental de detectar los eventos raros y valiosos.

1.3.1 Estrategias de mitigación

A Nivel de Datos (Remuestreo). El remuestreo (o *resampling*) consiste en modificar el conjunto de datos de entrenamiento para crear un conjunto más balanceado sobre el cual el modelo pueda aprender.

- **Sobremuestreo (Oversampling):** Aumenta el número de instancias en la clase minoritaria. El método más simple, el sobremuestreo aleatorio, simplemente duplica instancias, lo que puede llevar al sobreajuste. Una técnica mucho más avanzada y efectiva es **SMOTE** (*Synthetic Minority Over-sampling Technique*). SMOTE no duplica, sino que crea nuevas instancias sintéticas. Para cada instancia de la clase minoritaria, encuentra a sus k vecinos más cercanos y crea nuevas instancias a lo largo de las líneas que los conectan en el espacio de características. Esto genera un dominio de decisión más grande y robusto para la clase minoritaria.

- **Submuestreo (Undersampling):** Reduce el número de instancias en la clase mayoritaria. El método más simple, el submuestreo aleatorio, elimina instancias al azar. Su principal desventaja es el riesgo de descartar información potencialmente valiosa contenida en los datos de la clase mayoritaria que podrían ser importantes para definir el límite de decisión.

A Nivel de algoritmo (Ponderación de clases). Una alternativa al remuestreo es modificar el propio algoritmo de entrenamiento para que preste más atención a los errores cometidos en la clase minoritaria. Esto se logra mediante la **ponderación de clases** (*class weighting*). Se asigna un peso mayor a las instancias de la clase minoritaria en la función de pérdida del modelo. De esta manera, clasificar incorrectamente un planeta (clase minoritaria) incurre en una penalización mucho mayor que clasificar incorrectamente un falso positivo (clase mayoritaria). Muchos algoritmos en librerías como Scikit-learn, incluyendo 'RandomForestClassifier', permiten establecer este peso con el hiperparámetro '*class_weight = 'balanced'*'.

1.4 Validación de modelos y prevención del sobreajuste

El Objetivo de la generalización. El objetivo final de un modelo de aprendizaje supervisado no es tener un rendimiento perfecto en los datos con los que fue entrenado, sino **generalizar** bien a datos nuevos y no vistos. Un modelo que simplemente memoriza los datos de entrenamiento (sobreajuste) es inútil en la práctica. Por lo tanto, es fundamental emplear técnicas de validación robustas que den una estimación honesta de cómo se comportará el modelo en el mundo real.

1.4.1 Train-Test split

La regla de oro del Machine Learning es: **un modelo debe ser evaluado en datos que no ha utilizado para su entrenamiento**. Para cumplir con esta regla, la primera acción en cualquier proyecto es dividir el conjunto de datos completo en al menos dos subconjuntos independientes:

- **Conjunto de entrenamiento (Training set):** Típicamente, el 70 % u 80 % de los datos. Este es el conjunto que se le muestra al modelo para que aprenda los patrones, correlaciones y límites de decisión.
- **Conjunto de prueba (Test Set):** El 20 % o 30 % restante. Este conjunto se mantiene "guardado bajo llave durante todo el proceso de entrenamiento y ajuste de hiperparámetros. Sólo se utiliza una vez, al final, para obtener una evaluación final e imparcial del rendimiento del modelo entrenado.

Esta separación simple es el requisito mínimo para una evaluación honesta del poder de generalización del modelo.

1.4.2 La Robustez de la validación cruzada (Cross-validation)

Una sola división de entrenamiento/prueba, aunque esencial, tiene una debilidad: el rendimiento medido en el conjunto de prueba puede depender en gran medida de la suerte de qué instancias cayeron en esa división particular. Podría obtener una estimación demasiado optimista o pesimista del rendimiento del modelo.

La **Validación cruzada de K pliegues (K-fold cross-validation)** es una técnica mucho más robusta para estimar el rendimiento de un modelo. El proceso es el siguiente:

1. Se toma el conjunto de entrenamiento y se divide aleatoriamente en K subconjuntos de igual tamaño, llamados "pliegues" (*folds*). Un valor común es $K = 5$ o $K = 10$.
2. Se inicia un bucle que se repetirá K veces.
3. En cada iteración i (de 1 a K):
 - El pliegue i se reserva como el conjunto de validación temporal.
 - Los $K - 1$ pliegues restantes se combinan para formar el conjunto de entrenamiento temporal.
 - Se entrena un nuevo modelo desde cero utilizando los datos de entrenamiento temporales.
 - Se evalúa el rendimiento de este modelo en el pliegue de validación temporal y se guarda la puntuación (ej. Puntuación F1).
4. Al final del bucle, se tendrán K puntuaciones de rendimiento diferentes.
5. La estimación final del rendimiento del modelo es el **promedio** de estas K puntuaciones.

Este método utiliza todos los datos tanto para entrenamiento como para validación, y la estimación final es mucho más estable y menos dependiente de una sola división aleatoria. La desviación estándar de las K puntuaciones también da una idea de la estabilidad del rendimiento del modelo.

2 Proyecto y planificación

Esta sección delinea el marco estratégico y la hoja de ruta que seguí para el desarrollo de mi proyecto. Establecí objetivos claros y medibles, describí la metodología de trabajo y organicé las actividades secuenciales que estructuraron mi investigación, desde la concepción inicial hasta la obtención de los resultados finales.

2.1 Objetivos

Mi propósito en este proyecto fue diseñar, implementar y evaluar un sistema computacional robusto para la clasificación de candidatos a exoplanetas. Para alcanzar esta meta, definí un objetivo general y una serie de objetivos específicos.

Objetivo general. Desarrollar y comparar el rendimiento de dos modelos de aprendizaje automático supervisado (un Árbol de Decisión y un Bosque Aleatorio) para la clasificación automática de Objetos de Interés Kepler (KOIs) en las categorías de ‘CONFIRMED’, ‘CANDIDATE’ y ‘FALSE POSITIVE’, utilizando el archivo de datos público de la misión.

Objetivos específicos.

1. Realizar un análisis exploratorio de datos (EDA) exhaustivo sobre el conjunto de datos de KOIs para comprender la distribución de las variables, identificar correlaciones y visualizar las características distintivas de cada clase.
2. Diseñar e implementar un pipeline de preprocesamiento de datos robusto y reproducible, que incluya la limpieza, imputación de valores faltantes y selección de las características más informativas.
3. Entrenar un modelo de **Árbol de Decisión** y analizar su estructura para obtener reglas de clasificación interpretables.
4. Entrenar un modelo de **Bosque Aleatorio** como un clasificador de ensamble de alto rendimiento.
5. Evaluar y comparar el rendimiento de ambos modelos utilizando un conjunto de métricas de clasificación apropiadas para datos desbalanceados.
6. Utilizar el modelo de Bosque Aleatorio para identificar y cuantificar la importancia relativa de las diferentes características físicas en el problema de clasificación.
7. Finalmente, compararé todos estos resultados usando otra técnica la cual es redes neuronales, el cual esta siendo trabajado por mi compañero Salvador Adolfo Arroyo Tacuba.

2.2 Metodología de trabajo

Abordé el proyecto siguiendo una metodología secuencial estándar en proyectos de ciencia de datos, conocida como el Proceso de Ciencia de Datos en Equipo (TDSP) o metodologías similares, que garantizan un desarrollo estructurado y resultados reproducibles.

2.2.1 Identificación de actividades

Desglosé mi flujo de trabajo en cinco fases principales, cada una con sus propias tareas y entregables:

1. **Fase 1: Comprensión y adquisición de datos.** Esta fase inicial consistió en una revisión bibliográfica profunda para comprender el contexto astrofísico del problema, la misión Kepler y los fundamentos del método de tránsito. Luego procedí a la descarga del conjunto de datos del Archivo de Exoplanetas de la NASA.
2. **Fase 2: Análisis exploratorio y preprocesamiento.** Realicé un análisis estadístico y visual de los datos para entender sus propiedades. Tomé decisiones clave sobre la limpieza de datos, la eliminación de columnas y la estrategia de imputación, documentando cada paso.
3. **Fase 3: Características y modelado.** Diseñé el pipeline de preprocesamiento, incluyendo la selección y escalado de características. Luego procedí a la implementación, entrenamiento y ajuste inicial de los modelos de Árbol de Decisión y Bosque Aleatorio.

4. **Fase 4: Evaluación y discusión.** Evalué los modelos finales sobre un conjunto de prueba no visto, utilizando las métricas definidas. Generé las visualizaciones de resultados (matrices de confusión, gráficos de importancia de características) y procedí a la interpretación y discusión de los hallazgos.
5. **Fase 5: Documentación y reporte final.** Esta fase consistió en la redacción de este documento, sintetizando mi metodología, los resultados y las conclusiones del proyecto de manera clara y estructurada.

3 Materiales y métodos

Esta sección proporciona una descripción técnica detallada de todos los componentes que utilicé en el proyecto, desde el conjunto de datos y las herramientas de software hasta la metodología de tratamiento de datos y modelado.

3.1 Conjunto de datos

El conjunto de datos que utilicé es el **Catálogo Acumulativo de Objetos de Interés Kepler (KOI)**, correspondiente a la entrega de datos del cuarto trimestre de la misión (DR25). Lo descargué del Archivo de Exoplanetas de la NASA. El catálogo contiene 9,564 instancias (candidatos) y 49 características por instancia. La variable objetivo es `koi_disposition`, que clasifica cada instancia como ‘CONFIRMED’, ‘CANDIDATE’ o ‘FALSE POSITIVE’. Este conjunto de datos es un estándar en la comunidad para el desarrollo y la prueba de algoritmos de clasificación de exoplanetas debido a su tamaño, riqueza de características y la calidad de sus etiquetas.

3.2 Framework y Librerías de Software

Mi análisis fue realizado íntegramente en el ecosistema de Python 3, aprovechando su vasto conjunto de librerías de código abierto para la ciencia de datos.

3.2.1 Pandas

La librería **Pandas** fue mi herramienta fundamental para la manipulación de datos. La utilicé para cargar el archivo CSV en una estructura de datos bidimensional llamada `DataFrame`, que me permitió una limpieza, filtrado, transformación e inspección de datos eficiente. Todas las operaciones de preprocesamiento inicial, como la eliminación de columnas y el mapeo de etiquetas, las realicé con Pandas.

3.2.2 Scikit-learn

Scikit-learn fue mi framework de aprendizaje automático central para este proyecto. Me proporcionó todas las herramientas necesarias para mi pipeline de modelado:

- **Preprocesamiento:** `SimpleImputer` para manejar valores faltantes y `StandardScaler` para el escalado de características.

- **Selección de Características:** `SelectKBest` para la selección automática de las variables más informativas.
- **Modelado:** Las clases `DecisionTreeClassifier` y `RandomForestClassifier` para la implementación de los modelos.
- **Validación:** La función `train_test_split` para dividir los datos y herramientas como `classification_report` y `confusion_matrix` para la evaluación del rendimiento.
- **Automatización:** La clase `Pipeline` para encapsular todo el flujo de trabajo en un único objeto.

3.2.3 Matplotlib & Seaborn

Para la visualización de datos, utilicé las librerías **Matplotlib** (la base para la creación de gráficos en Python) y **Seaborn** (una interfaz de alto nivel construida sobre Matplotlib). Las empleé para generar los histogramas, diagramas de caja (box plots), mapas de calor de correlación y otros gráficos utilizados en el análisis exploratorio de datos y en la presentación de resultados.

3.3 Metodología

3.3.1 Pipeline en aprendizaje automático

En aprendizaje automático, un *pipeline* es una secuencia ordenada de transformaciones de datos seguida de un modelo final. Los datos de entrada atraviesan cada etapa de manera sucesiva. Matemáticamente, puede representarse como una composición de funciones:

$$P(X) = (f_m \circ f_{m-1} \circ \dots \circ f_1)(X), \quad (7)$$

donde $f_1, f_2, \dots, f_{m-1}$ corresponden a transformadores (imputación de valores faltantes, selección de características, normalización, etc.), y f_m es el estimador final encargado de la predicción.

Entrenamiento del pipeline Durante el ajuste (*fit*), cada transformador T_i aprende sus parámetros internos a partir de los datos de entrenamiento. Si $X^{(0)} = X$ se define como los datos iniciales, el proceso para $i = 1, \dots, m$ es:

$$\theta_i = T_i.\text{fit}(X^{(i-1)}, y), \quad (8)$$

$$X^{(i)} = T_i.\text{transform}(X^{(i-1)}). \quad (9)$$

Finalmente, el estimador E ajusta sus parámetros con los datos ya transformados:

$$\hat{\beta} = E.\text{fit}(X^{(m)}, y). \quad (10)$$

3.3.2 Ejemplo de transformaciones comunes

Imputación por la mediana Para una característica j , la mediana se define como:

$$\text{median}_j = \text{median}\{x_{ij} : x_{ij} \text{ observado}\}. \quad (11)$$

La imputación reemplaza cada valor faltante x_{ij} por dicha mediana:

$$\tilde{x}_{ij} = \begin{cases} x_{ij}, & \text{si } x_{ij} \text{ está observado,} \\ \text{median}_j, & \text{si } x_{ij} \text{ es faltante.} \end{cases} \quad (12)$$

Selección de características por información mutua La información mutua mide la dependencia entre una variable X_j y la clase Y :

$$I(X_j; Y) = \sum_{x_j} \sum_y p(x_j, y) \log \frac{p(x_j, y)}{p(x_j)p(y)}. \quad (13)$$

Cada característica recibe una puntuación $s_j = I(X_j; Y)$, y se seleccionan las k de mayor valor:

$$\text{selected} = \arg \text{topk}_j(s_j). \quad (14)$$

Estandarización Cada característica j se transforma de modo que tenga media cero y varianza unitaria:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad (15)$$

donde

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2}. \quad (16)$$

Prevención de *data leakage* Un aspecto crucial del pipeline es que cada transformador se ajusta exclusivamente con los datos de entrenamiento. Por ejemplo, las medias μ_j y desviaciones estándar σ_j utilizadas para estandarizar se calculan únicamente a partir del conjunto de entrenamiento. De esta forma, la transformación sobre el conjunto de prueba utiliza parámetros fijos:

$$z_{ij}^{\text{train}} = \frac{x_{ij}^{\text{train}} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}, \quad (17)$$

$$z_{ij}^{\text{test}} = \frac{x_{ij}^{\text{test}} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}. \quad (18)$$

Si, en cambio, se calcularan parámetros con todo el conjunto de datos (incluyendo prueba), se introduciría información no permitida y se produciría un sesgo optimista en la evaluación.

3.3.3 Tratamiento de los datos

Mi tratamiento de los datos siguió un protocolo riguroso para asegurar la validez de los resultados.

4.3.1.1. División en conjuntos de entrenamiento, validación y prueba. La regla fundamental del aprendizaje supervisado es evaluar un modelo en datos que no ha visto durante su entrenamiento. Para ello, dividí el conjunto de datos completo en un **conjunto de entrenamiento (70 %)** y un **conjunto de prueba (30 %)**. Esta división la realicé de manera estratificada utilizando el parámetro `stratify` en la función `train_test_split`. La estratificación garantiza que la proporción de cada clase ('CONFIRMED', 'CANDIDATE', 'FALSE POSITIVE') sea idéntica en ambos conjuntos, lo cual es crucial para evitar sesgos en la evaluación de datos desbalanceados. El conjunto de entrenamiento lo utilicé para todas las fases de construcción del modelo, incluyendo el ajuste del pipeline y la validación cruzada. El conjunto de prueba se mantuvo completamente aislado y lo utilicé una única vez al final del proceso para obtener una estimación final e imparcial del rendimiento de generalización del modelo. Hub

4.3.1.2. Datos desbalanceados. El conjunto de datos de KOIs está intrínsecamente desbalanceado: la clase 'FALSE POSITIVE' es significativamente más numerosa que las clases 'CANDIDATE' y 'CONFIRMED'. Este desbalance es un desafío, ya que un modelo ingenuo puede lograr una alta exactitud simplemente prediciendo siempre la clase mayoritaria. Para mitigar este problema, mi estrategia principal fue la **ponderación de clases (class weighting)**. Utilicé el hiperparámetro `class_weight='balanced'` en la implementación de mis modelos. Este modo ajusta automáticamente los pesos de las clases de forma inversamente proporcional a su frecuencia en los datos de entrenamiento. En la práctica, esto significa que mi modelo es penalizado mucho más fuertemente por cometer un error en una instancia de una clase minoritaria (como 'CONFIRMED') que por un error en una instancia de la clase mayoritaria, forzándolo a prestar más atención a los eventos raros y valiosos.

4.3.1.3. Selección de características. El rendimiento de los modelos depende críticamente de la calidad de las características de entrada. Mi proceso comenzó con una limpieza inicial, descartando columnas no informativas. Posteriormente, dentro del pipeline, apliqué una selección de características automática utilizando `SelectKBest` con la prueba estadística F de ANOVA (`f_classif`) para seleccionar las 30 características con la mayor dependencia estadística con respecto a la variable objetivo. Este paso me permitió reducir el ruido, disminuir la complejidad computacional y mitigar el sobreajuste. Las características finales utilizadas por mi modelo son, por tanto, un subconjunto de las variables físicas y de validación del catálogo KOI, seleccionadas por su relevancia estadística para la tarea de clasificación.

4.3.1.3. Dos formas de clasificación. Hubo dos formas de clasificación, el primer modelo "a" en el cual nuestro target para entrenar era binario, utilizando a CONFIRMED (1) y FALSE POSITIVE (0), con lo cual los CANDIDATE se eliminan por completo.

Para el segundo modelo "b", utilice un target multiclase. En este caso si se utilice las 3 clases que estaban en nuestros datos, con lo cual los CANDIDATE se mantienen como clase propia.

A partir de realizar lo anterior y aplicar los modelos, podemos pasar a ver los resultados obtenidos.

3.4 Resultados y discusión

Al utilizar pipeline para ver las características mas importantes, se obtuvieron las que se pueden ver en la siguiente lista:

Tabla 1: Características Ordenadas por puntaje de selección (Feature Importance)

Posición	Feature	Selection_Score
1	koi_prad	0.186604
2	koi_steff_err1	0.178987
3	koi_steff_err2	0.167278
4	koi_prad_err1	0.164017
5	koi_duration_err1	0.153230
6	koi_duration_err2	0.150499
7	koi_srad_err1	0.145116
8	koi_prad_err2	0.144563
9	koi_slogg_err1	0.130376
10	koi_period	0.118358
11	koi_slogg_err2	0.118269
12	koi_time0bk_err1	0.115868
13	koi_time0bk_err2	0.114778
14	koi_impact	0.112892
15	koi_depth	0.108618
16	koi_period_err2	0.105648
17	koi_period_err1	0.105553
18	koi_insol_err1	0.105057
19	koi_insol_err2	0.102547
20	koi_teq	0.100137

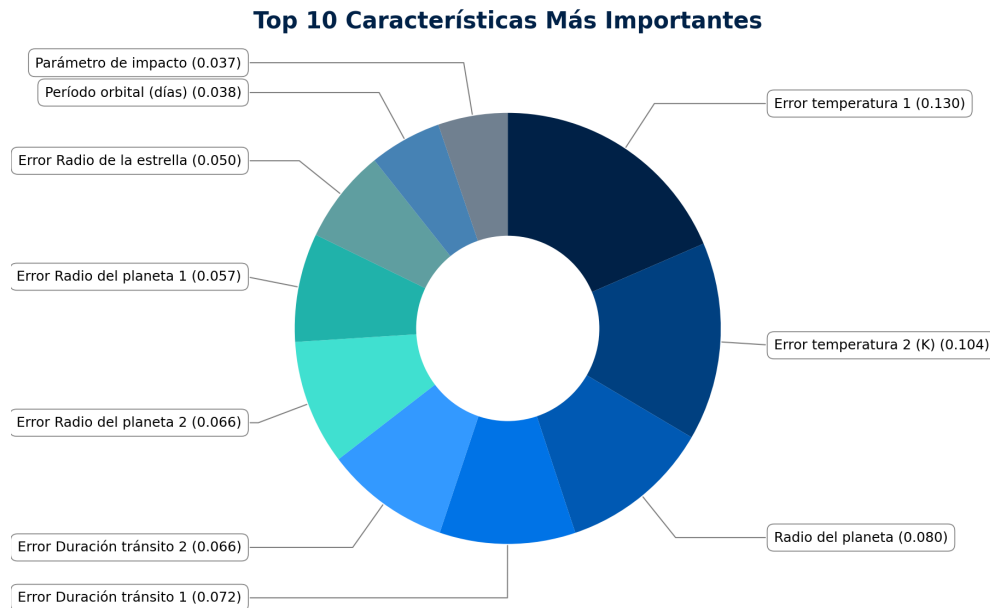


Figura 5: Diagrama de las 10 características más importantes

3.4.1 Forma binaria.

El primer paso que se hizo, fue eliminar la clase de CANDIDATES. Se hizo un ajuste de datos, donde elimine variables de metadatos o identificadores. Ahora, se aplicó la función de pipeline, con lo cual, pasamos de 34 variables a 20. Ya teniendo estos datos, pasamos a la parte de aplicar los modelos. Antes de empezar, como vimos, los modelos principales de este proyecto son los Árboles de decisión y Random Forest. Sin embargo, como no le asigne ningún valor a los argumentos de la función estos utilizarán el valor predeterminado que les asigna python. Los regularizados, los indiqué mientras desarrolle el trabajo, ya que vi muchos problemas al clasificar las clases. Para este modelo, primero visualizemos los valores de sus métricas obtenidas.

Modelo	CV Accuracy	CV Std	Train Accuracy	Test Accuracy	Overfitting Gap	F1-Score	Kappa	MCC
Árbol de Decisión (Normal)	0.8979	0.0015	1.0000	0.8961	0.1039	0.8955	0.7560	0.7563
Árbol de Decisión (Regularizado)	0.9104	0.0119	0.9422	0.9107	0.0315	0.9100	0.7895	0.7901
Random Forest (Normal)	0.9336	0.0100	1.0000	0.9362	0.0638	0.9353	0.8480	0.8500
Random Forest (Regularizado)	0.9266	0.0084	0.9504	0.9303	0.0201	0.9292	0.8335	0.8359

Tabla 2: Tabla comparativa de rendimiento de la forma binaria.

Veamos que el que mejor clasifico las clases, fue el Random Forest(normal), pero es exactamente el problema que vimos cuando estábamos desarrollando el modelo. Mi modelo tenía muy buenos valores al predecir las clases, sin embargo, existía un gap de overfitting muy grande, dicho con otras palabras, el modelo aprende demasiado bien los datos de entrenamiento, lo cual es malo ya que si aplicamos nuestro modelo a otros datos, los va a predecir muy mal. Para ello, están los modelos regularizados, los cuales evitan el overfitting en gran medida.

Sabiendo lo anterior, podemos darnos cuenta que el mejor modelo para aplicar en cualesquiera otros datos, no solo nuestro cumulative de kepler, es el Random Forest(regularizado). El valor del overfitting, es muy importante, pero podemos ver de forma más clara como clasificaron nuestras clases los modelos de la **Tabla 1.** a partir de las matrices de confusión.

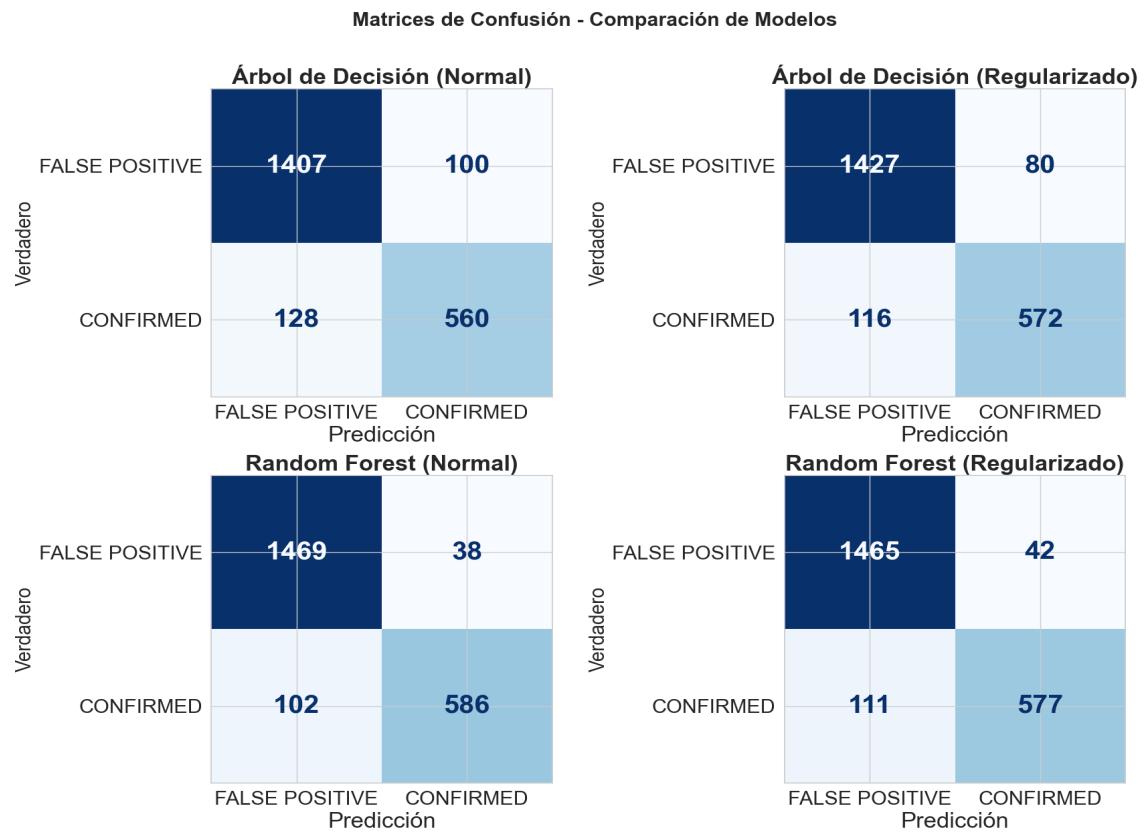


Figura 6: Matrices de confusión de los 4 modelos para la forma binomial.

Aquí es más facil ver lo que se mencionó anteriormente. En las siguientes imagenes, podemos ver las diferencias de overfitting que hay entre los dos modelos de random forest.

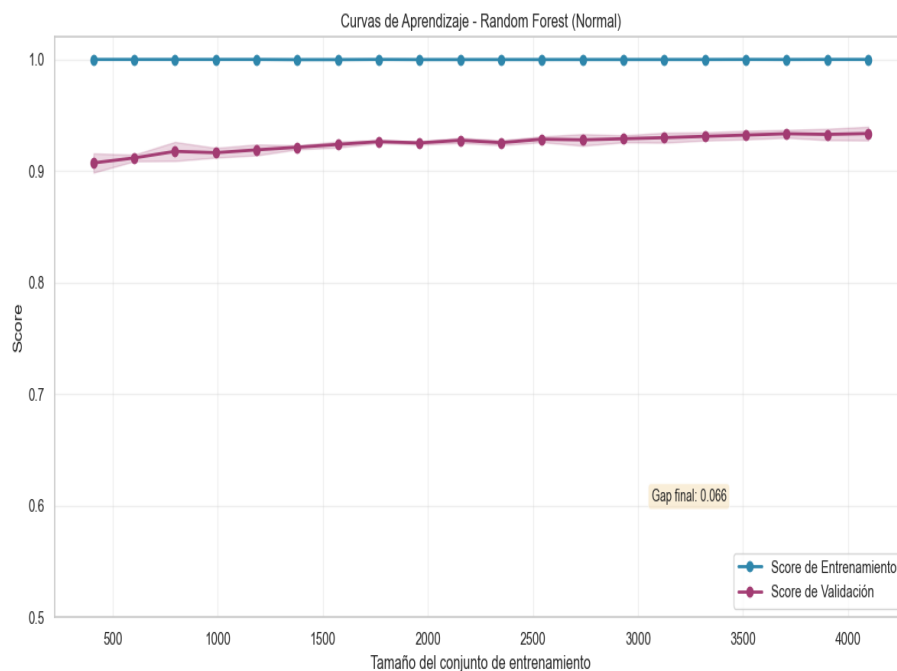


Figura 7: Gráfica del tamaño del conjunto de entrenamiento vs el score del modelo random forest(normal) para la forma binaria.

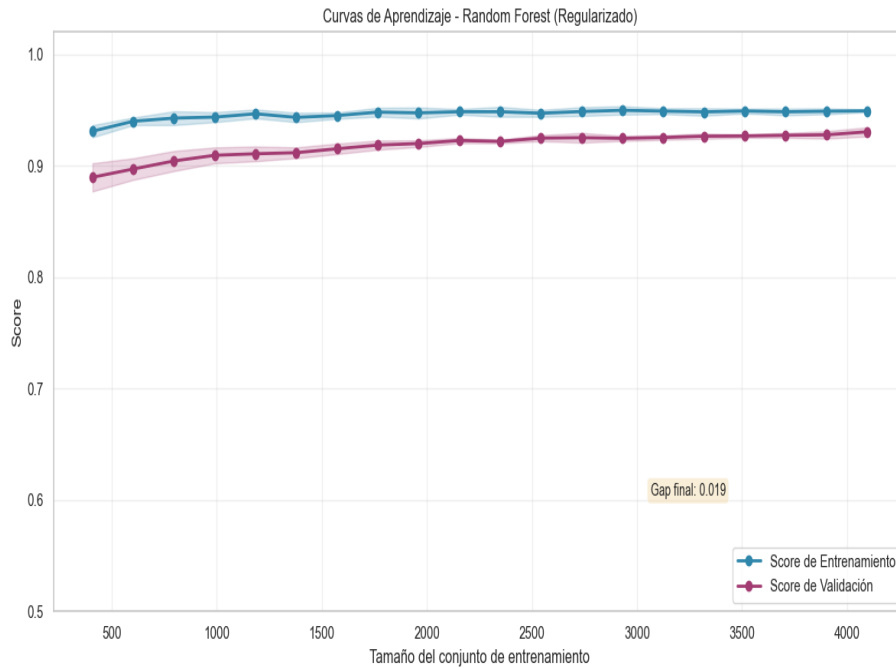


Figura 8: Gráfica del tamaño del conjunto de entrenamiento vs el score del modelo random forest(regularizado) para la forma binaria.)

A partir de lo que se desarrolló, podemos mencionar cosas que son importantes o que podrían mejorar nuestro proyecto en un futuro.

1. Los modelos regularizados muestran mejor generalización.
2. Random Forest generalmente supera a árboles individuales.
3. Considerar ensemble methods para mejorar el rendimiento.
4. Al no considerar los candidatos, vemos que la precisión de los modelos es muy alta.

3.4.2 Forma multiclase.

Para esta parte, se hizo lo mismo que anteriormente se mencionó. Con lo cual, pasaré a ver los valores de las métricas.

Modelo	CV Accuracy	CV Std	Train Accuracy	Test Accuracy	Overfitting Gap	F1-Score	Kappa	MCC
Árbol de Decisión (Normal)	0.6672	0.0074	1.0000	0.6652	0.3348	0.6677	0.4526	0.4529
Árbol de Decisión (Regularizado)	0.7133	0.0069	0.7823	0.7132	0.0691	0.7112	0.5273	0.5275
Random Forest (Normal)	0.7531	0.0060	1.0000	0.7564	0.2436	0.7443	0.5893	0.5937
Random Forest (Regularizado)	0.7493	0.0032	0.8177	0.7512	0.0665	0.7353	0.5799	0.5860

Tabla 3: Tabla comparativa de rendimiento la forma multiclase.

Nuevamente, el que clasifica mejor las clases, es random forest(normal), pero lo hace con un overfitting muy grande. Con lo cual, al ver los valores de las métricas y el overfitting, vemos que el mejor modelo es, de nuevo, random forest(regularizado). A continuación, también veremos sus tablas de confusión y la diferencia de overfitting de los 2 modelos de mejor precisión, que fueron los random forest.

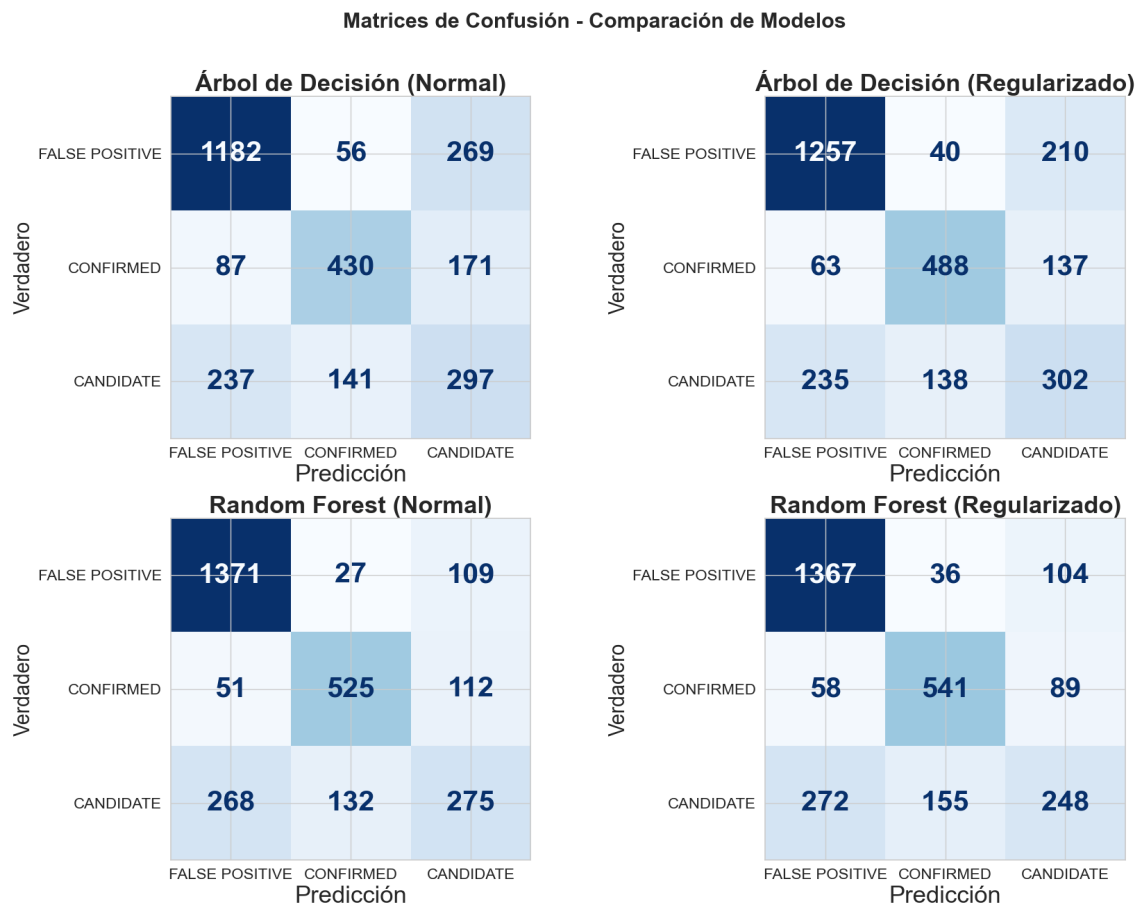


Figura 9: Matrices de confusión de los 4 modelos para la forma de multiclase.

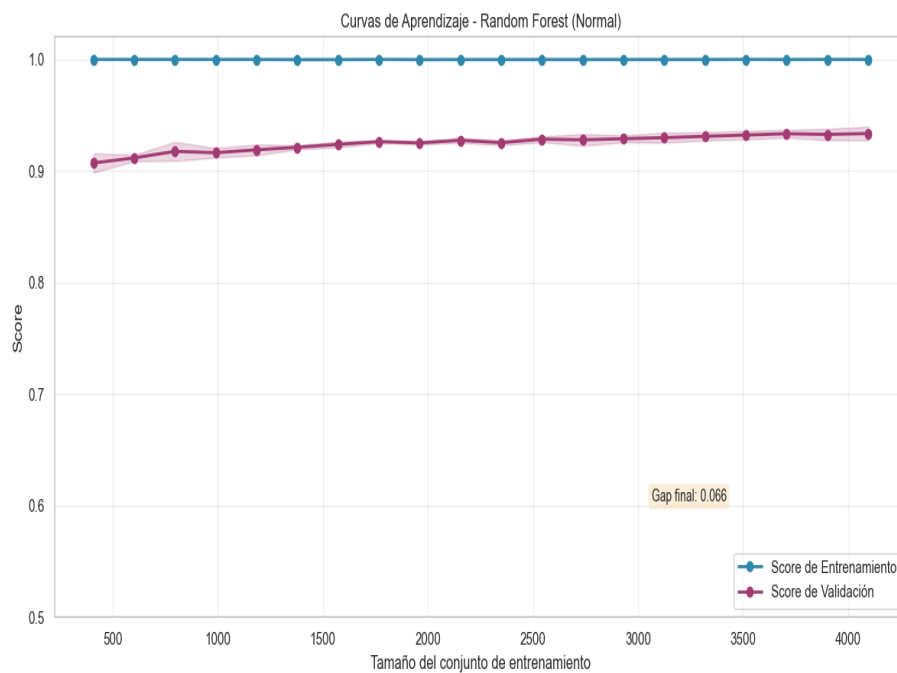


Figura 10: Gráfica del tamaño del conjunto de entrenamiento vs el score del modelo random forest(normal) para la forma de multiclase.

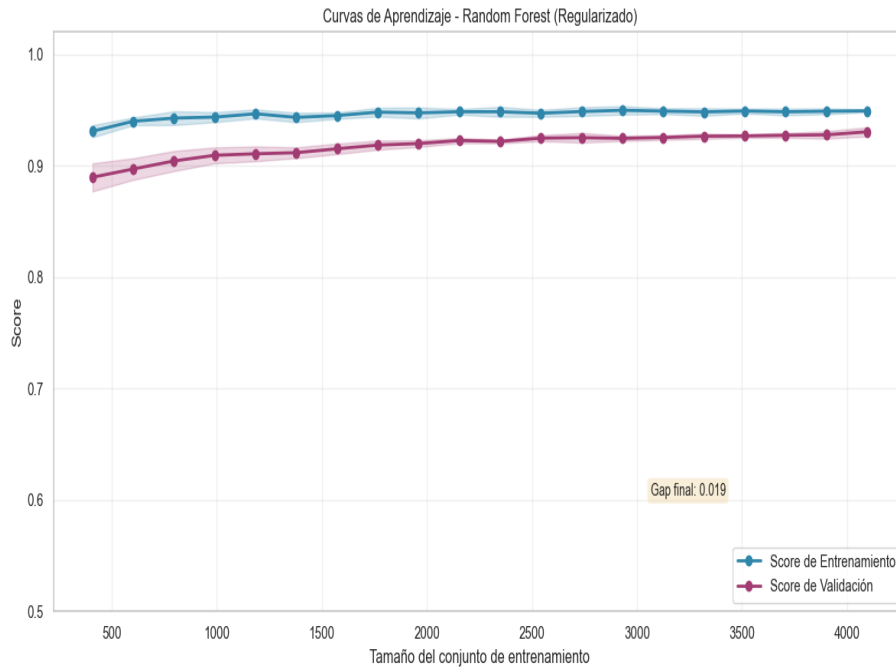


Figura 11: Gráfica del tamaño del conjunto de entrenamiento vs el score del modelo random forest(regularizado) para la forma de multiclase.

Vemos que al incluir a los CANDIDATE como una clase propia, la precisión de los modelos es más baja.

3.4.3 Resultados finales

A partir de todo lo anterior, vemos nuevamente que el machine learning es una de las herramientas más potentes y usadas en el campo de la astrofísica y en el análisis de datos de telescopios. En nuestro caso, el mejor modelo obtenido para clasificar estrellas, fue Random Forest. Random Forest suele ser mejor para clasificar estrellas porque combina muchos árboles de decisión que votan por la clase final, lo que lo hace muy robusto al ruido y a errores de medición, comunes en datos astronómicos. Además, puede manejar muchas variables a la vez, como magnitudes en distintos filtros, colores y metalicidad, sin requerir preprocesamiento complejo, y reduce el sobreajuste al no depender de un solo árbol. Esto permite clasificar correctamente incluso cuando las clases están desequilibradas o los datos contienen incertidumbres, lo que lo hace ideal para tareas como distinguir tipos espectrales o estados estelares a partir de observaciones telescópicas.

Veamos los valores obtenidos para redes neuronales, los cuales fueron desarrollados por mi compañero Salvador Adolfo Arroyo tacuba, los cuales están en la siguiente tabla junto a mis valores.

Tabla 4: Comparación de rendimiento de modelos de clasificación

Modelo	Modelo A		Modelo B	
	Accuracy	F1-score	Accuracy	F1-score
Random Forest (Normal)	93.6 %	93.5 %	75.6 %	74.4 %
Random Forest (Regularizado)	93.0 %	92.9 %	75.1 %	73.5 %
Red Neuronal (Normal)	92.8 %	92.8 %	72.9 %	70.8 %
Red Neuronal (Regularizada)	92.3 %	92.2 %	72.8 %	70.2 %
Árbol de Decisión (Regularizado)	91.0 %	91.0 %	71.3 %	71.1 %
Árbol de Decisión (Normal)	89.6 %	89.5 %	66.5 %	66.8 %

4 Conclusión

Los resultados experimentales, resumidos en la Tabla 4,, demuestran de manera concluyente la superioridad del algoritmo **Random Forest** para la tarea de clasificación de exoplanetas. El modelo **Random Forest (Normal)** alcanzó las métricas más altas de *Accuracy* (**93.6 %**) y *F1-score* (**93.5 %**) en el Escenario A, y mantuvo su liderazgo en el Escenario B (*Accuracy*: 75.6%; *F1-score*: 74.4%). Sin embargo, en el modelo binario, el *Bosque Aleatorio Regularizado* alcanzó una *accuracy* del **93.0 %** y un *F1-score* del **92.9 %**. En el modelo multiclase, obtuvo una *accuracy* del **75.1 %** y un *F1-score* del **73.5 %** pero con un gap menor de overfitting, siendo así este el mejor modelo. Esta consistencia supera significativamente a las Redes Neuronales y al Árbol de Decisión simple, confirmando que el método de ensemble de los bosques aleatorios ofrece una robustez y una capacidad de generalización óptimas, minimizando el sobreajuste y estableciéndose como la técnica de Machine Learning más efectiva, de las que se vieron en este proyecto, para este conjunto de datos de la misión Kepler.

Bibliografía

1. Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
2. Borucki, W. J., Koch, D. G., Basri, G., Batalha, N., Brown, T. M., Caldwell, D., ... & DeVore, E. (2010). Kepler planet-detection mission: introduction and first results. *Science*, 327(5968), 977-980.
3. Bryson, S. T., Jenkins, J. M., Klaus, T. C., Twicken, J. D., & Clarke, B. D. (2020). The Kepler science operations center pipeline: An overview and recent enhancements. *Publications of the Astronomical Society of the Pacific*, 132(1013), 074502.
4. Fulton, B. J., Petigura, E. A., Howard, A. W., Isaacson, H., Marcy, G. W., Cargile, P. A., ... & Johnson, J. A. (2017). The California-Kepler Survey. III. A Gap in the Radius Distribution of Small Planets. *The Astronomical Journal*, 154(3), 109.
5. Jenkins, J. M., Caldwell, D. A., Chandrasekaran, H., Twicken, J. D., Bryson, S. T., Dunham, E. W., ... & Li, J. (2010). Overview of the Kepler science processing pipeline. *The Astrophysical Journal Letters*, 713(2), L87.
6. Koch, D. G., Borucki, W. J., Basri, G., Batalha, N. M., Brown, T. M., Caldwell, D., ... & Dunham, E. W. (2010). Kepler mission design, realized photometric performance, and early science. *The Astrophysical Journal Letters*, 713(2), L79.
7. Mandel, K., & Agol, E. (2002). Analytic light curves for planetary transit searches. *The Astrophysical Journal Letters*, 580(2), L171.
8. Mullally, F., Thompson, S. E., Coughlin, J. L., Burke, C. J., & Rowe, J. F. (2015). The California-Kepler Survey. VI. The Planet Radius Gap. *The Astronomical Journal*, 150(6), 195.
9. Quinlan, J. R. (1986). Induction of decision trees. *Machine learning*, 1(1), 81-106.
10. Wikipedia. (s. f.). Kepler (telescopio espacial). *Wikipedia, la enciclopedia libre*. Recuperado el 1 de octubre de 2025, de "[https://es.wikipedia.org/wiki/Kepler_\(telescopio_espacial\)](https://es.wikipedia.org/wiki/Kepler_(telescopio_espacial))"